

Predicting NBA Game Outcomes Through Machine Learning Techniques

Rymer Popovich
College of Information, Sciences, & Technology
Pennsylvania State University
State College, US
rfp5404@psu.edu

Andrew Bauer
College of Information, Sciences, & Technology
Pennsylvania State University
State College, US
ajb8391@psu.edu

I. ABSTRACT

We propose a data-driven project that aims to develop a machine learning model predicting the outcomes of NBA games using advanced team and performance-based statistics. By building upon prior studies, our model incorporates features engineered from historical game data, including an efficiency index and rolling averages that are calculated over x recent games. This allows us to capture team momentum without worrying about data leakage. Using a Random Forest Classifier, our model achieves a competitive prediction accuracy and also includes optimal time windows to enhance the performance even further. We utilize data preprocessing, feature engineering, and model optimization techniques to develop and train a model that can analyze key efficiency and shooting metrics to identify the more likely winning team. By building off of prior work relating to sports prediction models, this study introduces new feature sets and optimizes time window parameters that will improve prediction accuracy. This contributes to a user-friendly prediction simulator that we created to demonstrate the real-world application potential of this model.

By integrating two other models, XGBoost and K-nearest Neighbors, we have opened the opportunity to compare the results between the three models. Our highest performing model was the K-nearest Neighbors with an accuracy of about 64.4%. Our second highest performing model was the XGBoost model with an accuracy of about 63.7%. Finally, our lowest performing model was actually the Random Forest model with an accuracy of about 63.3%. Although all three models performed relatively equal in terms of their metrics, the XGBoost model allowed for the most parameter tuning. There could very well be more improvements that would require further testing regarding the parameters. These will be discussed in the future work section at the end of the paper. In terms of real-world accuracy, a 64% could be beneficial towards the betting industry if interpreted properly. We will continue to integrate new ideas into this model to create the most reliable NBA prediction model possible.

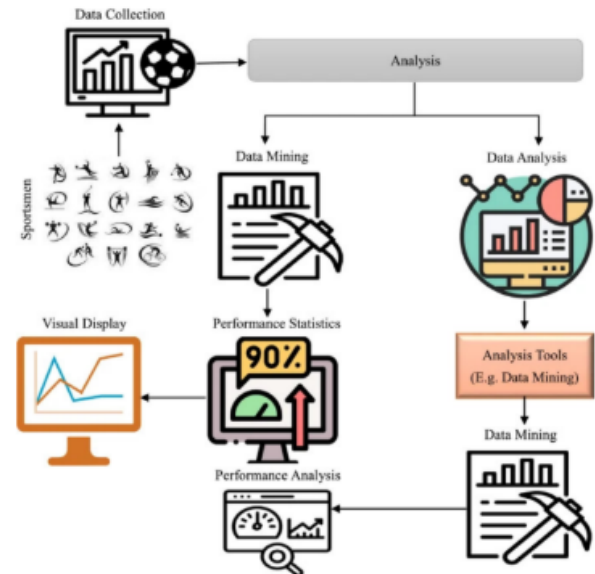


Figure 1: Flow diagram of match result prediction using machine learning [3]

II. INTRODUCTION

In this document we will be discussing and reviewing three research papers that have inspired us to take on this challenge. Two of the research papers will be related to predicting winners of NBA games, and the other paper will be an overview of AI in sports. Once we have gone over these research papers, we will talk about how we recreated the results of our first paper, our “parent paper”, and what we changed to try and improve on that initial model.

Predicting the outcomes of NBA games can present a challenge due to the nature of sports and the noisy data and numerous variables that influence the results. Trying to predict an NBA game, or any sports game, without the use of outside sources is extremely difficult because of how many factors go into deciding wins and losses. Our model aims to be simple enough for everyday fans to use and accurate enough for people who work in sports to use. It will be simple to interpret because all we are predicting is wins and losses, not player performances on a given night or season. Accurate game predictions are very highly valued, specifically in the sports betting industry. Sports betting industries invest heavily into creating prediction models that are accurate. The issue with traditional approaches is that

they depend too much on simple metrics. This project combines our passion with sports and data science expertise to create a model that can effectively and accurately predict NBA game outcomes by using advanced statistical features. One of our goals with this model is to allow it to be easily used by all users that may not have much technical knowledge. We aim to enhance prediction accuracy through feature engineering and optimal time window (OTW) selection. The dataset we will be using contains 33316 NBA games that occurred between 2010-2024.

The research paper that we will analyze first [1] is going to be our parent paper for our document, and is also the paper that we replicated the results for later in the document. This paper is titled “Predicting outcomes of NBA games through machine learning techniques”. This research paper is trying to find the best OTWs, features, and models to create the most accurate NBA game predictor. The researchers also experimented with an efficiency index that incorporates positive and negative stats and gives them a weight to compare teams with.

The second research paper [2] that we will be analyzing is “Predictive analysis of NBA game outcomes through Machine Learning” which is very similar to the first research paper, however they believe that they can create a strong model using basic statistics and machine learning models without deriving their own features. This paper helped us in our work by showing us that it is not always necessary to create advanced features based of the features that already exist because you can still achieve a strong accuracy.

The third research paper [3] that we will be discussing is the “Overview of the Application of Artificial Intelligence in Sports Events”. We found this paper interesting because it shows a broader scope of using artificial intelligence in sports and expands on the possible outcomes of its use. Our two previous papers also referenced using player tracking and motion sensors to improve their models in the future and this paper touched upon that in their research with badminton swings. It was nice to work with a paper that was not predicting the outcomes of games, rather showing what could be done in the future and how AI can help a lot of people in sports.

III. LITERATURE REVIEW

A. Predicting outcomes of NBA games through machine learning techniques [1]

This paper has consistently gained increasing attention with the growth of sports analytics and AI. A study done by Horvat et al. developed a machine learning algorithm that used a publicly available NBA dataset which includes 6567 games from 2013-2018 in order to predict game winners. They achieved a peak accuracy of 84% using a Random Forest classifier. Although their approach was very effective, it relied too heavily on a smaller dataset. In their paper they derived an NBA efficiency index which inspired us in our project to build upon the framework that they created by incorporating some more advanced feature

engineering. This is inspired by their methodology and aims to further improve prediction performance. Another key inspiration from Horvat et al.’s study involves optimal time windows (OTW). These allow for the balance of recent performance trends with the avoidance of data leakage from future information. By selecting a specific window (ex: 10 games), models are able to capture team momentum and ensure that only past games can influence its predictions. This technique was implemented in our project’s feature engineering phase. We created rolling averages over 10-game periods to inform team efficiency indices and shooting metrics.

B. Predictive analysis of NBA game outcomes through Machine Learning [2]

This paper by Junwen Wang uses statistics from 25,797 NBA games spanning from 2004-2020 [2]. This paper built upon traditional statistical models which were underperforming due to all the noisy statistics from games. They believe that advanced machine learning models need to be leveraged to identify key performance factors. Their dataset used inputs like field goal percentage (FG%), free throw percentage (FT%), three point percentage (3P%), assists (AST), rebounds (REB), and team points to get a binary output, wins and losses. They found that random forest gave them an accuracy of 83.8% with the most predictive factor being FG%. They concluded that teams should focus on efficiency when strategizing. One area that this paper lacks is a use of advanced statistics such as true shooting percentage (TS%) or effective field goal percentage (eFG%). They said that teams should focus on efficiency, however they are just using a base FG% which weighs threes and twos the same. Using a more advanced metric may boost their accuracy scores and allow for more precise predictions. This project reinforced our idea of using a random forest model because they also achieved their peak accuracy when using one.

C. Overview of the Application of Artificial Intelligence in Sports Events [3]

In this paper Huali Dong shows the wide applicability and potential of artificial intelligence. In his research, he discusses how AI can be utilized for outcome prediction, performance enhancement, injury prevention, logistics improvement, and fan engagement. Dong highlights the importance of combining real-time data with historical data to create predictive models that are able to respond to the unpredictable nature of sports. The study stresses challenges such as data quality, interpretability of AI models, and domain expertise to ensure effective insights. Overall, this work shows AI-driven sports analytics as an opportunity to improve decision-making for coaches, athletes, and sports organizations while enhancing the experience for fans. This paper is one that we wanted to look at because of the future work of our previous papers. They referenced tracking player movements and other metrics to predict player and team performance and this paper used player tracking movements when working with badminton players to visualize swing patterns. When working with player tracking data in the NBA you may be able to see how a

shooter performs when a defender is nearby when contesting a shot or if they are far away. This would help benefit shooting efficiencies, which our previous papers stated are important for predicting NBA game outcomes. This paper gives an insight into a potential future of AI and machine learning in sports and it is intriguing to see what may come out of new data.

Our study complements all three of these works by focusing on NBA team and performance-based statistics, utilizing feature engineering techniques such as rolling averages over optimal time windows to avoid data leakage. Additionally, we included a home-court advantage factor and created an easily interpretable model that is suited for public use. Our data pipeline; data preprocessing, feature enhancement, and model optimization, aligns with sports analytics research while contributing enhancements. From *A*, we took inspiration from the optimal time windows as they allowed for us to integrate momentum into our model while also avoiding any data leakage concerns. Utilizing rolling features not only improved our accuracy, but allowed us to tune the number of games to test the model over different spans of games. From *B*, we took inspiration from Junwen Wang’s ideology of advanced statistics and how crucial they are in the model training and development process as well as the importance of choosing a reliable and consistent model. In his study, Wang utilized the Random Forest classifier and predicted at a 83.8% accuracy. He also identified the most predictive feature (FG%). With these conclusions, we are able to use a Random Forest classifier in our model along with creating a feature importance chart to identify the most influential factors to the game outcomes. From *C*, we took inspiration from Huali Dong’s conclusion of AI-driven sports analytics becoming an opportunity to improve decision-making for coaches, athletes, and sports organizations. This is referring to the application possibilities of our model. We have developed a very user-friendly and interpretable prediction simulator that can utilize our model to display the model’s predictions in an insightful manner that is meaningful to the consumer.

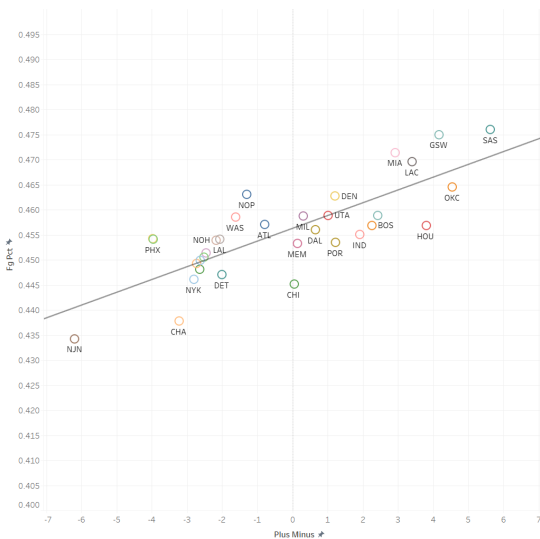


Figure 2: Correlation between most predictive feature (FG%) and Plus Minus (directly correlates to wins)

IV. CONTRIBUTIONS

For our CONTRIBUTIONS section we will be going over our original plan to build an NBA predictor model. In this section we will also discuss what challenges that gave us and how we plan on fixing those problems to improve our model.

A. Original Plan for Implementation

The initial plan we had for our model consisted of many of the same ideas from the first research paper we read. Our “parent paper” did not have a Github that we could reference so we had to replicate their code and results from scratch. To find our dataset, we searched on Github and found a dataset that included NBA games over multiple seasons that we referenced previously. As discussed in our [III] LITERATURE REVIEW, we were planning to implement the efficiency index to summarize the nba statistics into fewer and more easily interpretable features. We had also decided that we were going to utilize the optimal time windows in the form of rolling features. This would allow us to avoid any data leakage. It would also allow for the predictions to stay consistent and team momentum to be factored into the predictions. The third key piece from the first research paper that we planned to implement was the Random Forest Classifier. We had noticed that it predicted at one of the highest accuracies out of all the models they tested. It was also a relatively simple model which allows for easy hyperparameter tuning. Our goal was to split the data 70/20/10 on the train/test/validation sets.

B. What Happened?

After the planning phase, our initial implementation began. We utilized R-Studio for our data preprocessing and model development. Once these phases were completed and debugged thoroughly, we were able to move into the testing phases of our model. During the first attempt with our Random Forest model, we noticed that the accuracy was significantly lower than that of the research paper. We assumed that our recreation of the code must not have been up to par, however, after reevaluating, we started to experiment with tuning our parameters. By increasing the `n_estimators` that the Random Forest Classifier was using, we had noticed a significant increase in the model's prediction accuracy. Although the accuracy was improving, the runtime was becoming obnoxiously long. The more parameters that we tuned, the longer it would take R-Studio to debug and run the code. Eventually this became an issue and we had to develop a new plan.

C. New Implementation Plan

As mentioned above, R-Studio was not going to cut it anymore. So we explored other options for IDEs. After much discussion, we landed on using Visual Studio Code, a very well-known IDE with vast library support and extension capabilities. We had also decided to switch the programming language from R to Python as it is a more universal language with extensive library support. We would dedicate about a day or so in converting the R code into Python and configuring the environment. Visual Studio also includes Microsoft Copilot implementation, which would

serve as a helpful tool for troubleshooting and debugging. In regards to the development phase, we would be continuing with the Random Forest Classifier while also integrating an XGBoost model and K-nearest Neighbors model. This would give us the capability of comparing the models to find the most accurate of the three. The last portion of our plan consisted of creating an easy to use user interface that would make predictions based on the predictions from the models and output to the terminal in well-formatted and easy to interpret messages. Thus finishing the planning phase for our final implementation and commencing the actual integration.

V. IMPLEMENTATION

In the IMPLEMENTATION section, we will be discussing how we worked with the data, and our previous recreation of the “parent paper” model to improve our model and achieve our highest accuracy. We will also discuss the prediction simulator that we created to implement the model and allow basketball fans to predict any matchups they can think of.

A. Data Preprocessing

Our first step in the data preprocessing stage consisted of stating our object: predicting NBA game outcomes using game matchup data. Our data sources consisted of simple and advanced NBA team statistics such as PTS, REB, TOV, etc. This dataset was publicly available on Github. The data was already mostly tidy and there was very little transforming necessary. We imported the data by reading the raw CSV file into our environment. Then we were able to start working on the data. Firstly, we needed to define which team was the home team based on whether the matchup contained “vs” or “@”. Obviously, in a TEAM @ TEAM scenario, the second team would be the home team, and vice versa for vs. With that done, we could start with our feature engineering where we defined our advanced features: MissedFG, MissedFT, TS, TS_Pct, FG_Eff, RebRatio, and EffIndex. All of these features are advanced NBA statistics besides EffIndex. That feature contains all of the simple statistics weighed at different values depending on how influential they are towards the result of a game. The positive stats such as PTS, REB, AST, STL, and BLK were first added together. Then we took into account the negative stats such as MissedFG, MissedFT, and TOV which were then subtracted from the positive statistics. This together gave us our efficiency index. With our efficiency index created and features defined, we could implement our optimal time windows by creating a rolling series and applying it to those features. After much experimentation, our final window became 50 games. We found that this was just enough games to allow for team momentum and to keep the predictions consistent, but also not too many games where we would have to be concerned about data leakage. With our rolling features now created, we would check the data for any missing values, drop them, and sort the remaining data. Now we were able to apply our 70/20/10 split cutoff date from the rolling features onto the data. Since the data was successfully sorted, transformed and split, we could now move into our model development.

$$TS\% = \frac{(0.5 \cdot FTM) + 2PM + (1.5 \cdot 3PM)}{FGA + (0.44 \cdot FTA)} = \frac{0.5 \cdot PTS}{FGA + (0.44 \cdot FTA)}$$

Figure 3: True Shooting Percentage Formula

B. Machine Learning Models

In our “parent paper” they used many machine learning models in their research, however the three that we saw achieve the most success were the random forest model, the XGBoost model, and the K-nearest neighbors model. We believe that these three models would be most likely to give the most consistent results.

a. Random Forest Model

The first model that we built was our random forest model. This is the model that we originally replicated from our “parent paper” and built in R-studio before moving it over to Visual Studio Code. Moving it to Visual Studio code allowed us to run the model faster so we could test more hyper-parameters to achieve the highest accuracy. This model would also serve as the baseline for any future models, so we wanted to make it as strong as possible.

We started by creating a function ‘make_matchup_df’ that creates a simple feature matrix from the games data, and we made sure for all of the feature columns that we were using the rolling features from the past 50 games. This would ensure that we were not comparing teams from multiple years worth of data because teams can change a lot over time so we wanted to make sure that we are comparing the most current versions of the teams in our model. We then created the feature vectors which would take the difference in each feature from team 1 and team 2 to see where each team is better and worse for all of our statistics so the model can compare them later. We then made sure to append a 1 to the feature vector if there is a ‘vs’ in the “MATCHUP” column to ensure the model knows which team is home, and a 1 if there is a ‘W’ in the ‘WL’ section so the model knows which team won. We used this function to create the training, testing and validation dataset.

For the random forest model itself, we found that using 300 n estimators would give us the highest accuracy. Overall, this model gave us a good baseline for our accuracy and we hoped to see an improvement in the other two models.

```
Model Evaluation:
Training set size: 9320 samples
Validation set size: 2330 samples
Accuracy on validation set: 0.633
```

Figure 4: Model Evaluation of Random Forest Model

b. XGBoost Model

The XGBoost model was the next model that we created because it achieved a relatively similar accuracy to the random forest model in papers [1] and [2]. We used the same ‘make_matchup_df’ function in this model so now all we had to do was make the model itself. The hyper-parameters that gave us the best accuracy for this model were 200 n estimators, a max depth of 6, a learning rate of .1, a subsample of .8, a column sample by tree of .8, and an evaluation metric of ‘logloss’. This model showed us that the most important feature is which team is home, followed by the efficiency index, and then TS%.

```
Model Evaluation:
Training set size: 9320 samples
Validation set size: 2330 samples
Accuracy on validation set: 0.637

Classification Report:
              precision    recall  f1-score   support

     0       0.64       0.64       0.64       1164
     1       0.64       0.64       0.64       1166

 accuracy          0.64          0.64          0.64       2330
 macro avg         0.64          0.64          0.64       2330
weighted avg         0.64          0.64          0.64       2330
```

Figure 5: Model Evaluation and Classification Report of XGBoost Model

c. K-nearest Neighbors Model

The final model that we built was the k-nearest neighbor model. We chose this model because it gave the highest accuracy to the ‘parent paper’ and we felt that we could achieve a better accuracy as well. We still used the same ‘make_matchup_df’ as the other two models. We wanted to test multiple k-values so we tested a range from 3-60 and found that k=35 would give us the highest accuracy. We then used this k value in the model and set the weights by distance, the algorithm to ‘auto’ and the metric to ‘euclidian’.

```
Model Evaluation:
Training set size: 9320 samples
Validation set size: 2330 samples
Accuracy on validation set: 0.644

Classification Report:
              precision    recall  f1-score   support

     0       0.65       0.63       0.64       1164
     1       0.64       0.65       0.65       1166

 accuracy          0.64          0.64          0.64       2330
 macro avg         0.64          0.64          0.64       2330
weighted avg         0.64          0.64          0.64       2330
```

Figure 6: Model Evaluation and Classification Report of K-Nearest Neighbors Model

C. Prediction Simulator

With the development of the models being completed, we technically had our results. However, the average person would not be able to interpret the metrics that the model was outputting (AUC, F1, etc.). So we came up with an idea for a friendly user-interface that would be easily interpretable. We called it the Prediction Simulator. This script loads the predictive model and rolling_averages dataset that were used to train the model. The core idea of this simulator is based on predicting the outcomes based on the differential of the teams statistics combined with the home-court information. The actual function of the script goes as follows: Firstly, the code checks whether both team input abbreviations exist in the team averages index that it creates. It then selects the features from the rolling_averages dataset that it will use to predict, where it will retrieve each team's statistics and compute the feature differences. This data will then be reshaped into a 2D array with a single row. It will then utilize the mode chosen to create its prediction. When a user runs this script, they will be prompted to enter two teams for a hypothetical matchup and to decide which team is the home team. During this prompt, the user will receive a neatly formatted table of all available teams to choose from. [SEE FIGURE 7] Finally, in terms of the output structure of this prediction simulator, a dictionary with two keys will be returned. These keys are the winner and the probability of team 1 winning. This script will keep running on a loop until the user inputs the letter “q” and it will come to a halt.

```
Available teams for prediction:
ATL  BKN  BOS  CHA  CHI  CLE
DAL  DEN  DET  GSW  HOU  IND
LAC  LAL  MEM  MIA  MIL  MIN
NJN  NOH  NOP  NYK  OKC  ORL
PHI  PHX  POR  SAC  SAS  TOR
UTA  WAS

Enter first team abbreviation (or 'q' to quit):
```

Figure 7: Prediction Simulator available choices for prompting

VI: RESULTS

After building all of our models and finding the highest accuracy for each of them, we wanted to compare the results of all of them between each other to show where they performed better and which features they deemed most important for predicting the outcomes of games. We created a Python script and used matplotlib to created the visualizations after adding the models to the Python script.

Figure 8 shows us a comparison between the accuracies of the three models that we created. Random Forest was the first model that we built which gave us an accuracy of 63.3%, and we only improved the accuracy from there. The second model that we built, XGBoost, gave us an accuracy

of 63.7% after tuning the parameters. The last model that we built, KNN, gave us our highest accuracy at 64.4%.

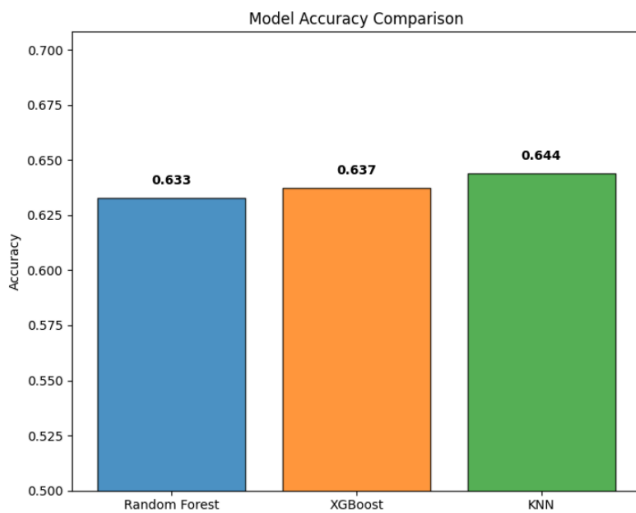


Figure 8: Comparison of accuracies between our models

In regards to the overall performance of the models, they were fairly consistent across the board. As you can see in Figure 9 below, all three models were within 0.01-0.02 units of each other within each metric. The accuracy of all three models ranged between 0.63 and 0.65, the AUC ranged between 0.68 and 0.69, and the precision ranged between 0.63 and 0.65. With the recall, there was a much larger disparity of scores compared to the other three metrics with the range falling between 0.62 and 0.66. Despite this range of recall scores, across the board, these values were very consistent and we are happy with the results.

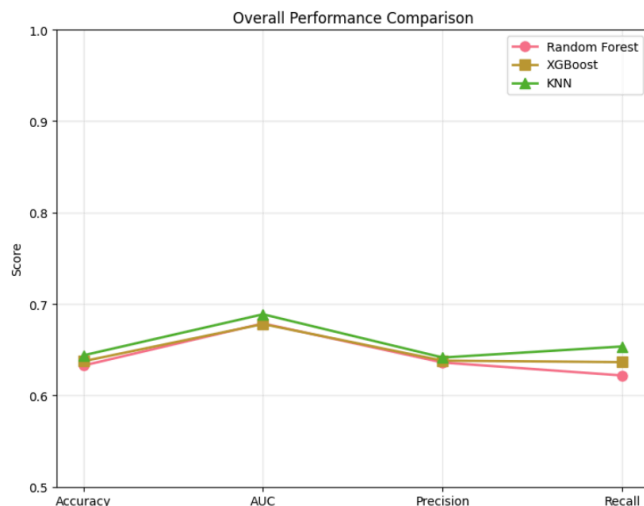


Figure 9: Overall performance comparison between models

We evaluated the importance of each feature in regards to each of the three models so that we could understand which features had the most influence on the predictions. Random Forest and XGBoost use traditional feature importances since they are tree-based models. However, KNN is not a

tree-based model, therefore we were required to use a permutation importance instead to find the feature correlation. When comparing the feature importance between the three models we noticed a few common trends. Firstly, the efficiency index was very influential on the KNN model's predictions, however, it didn't have nearly as much influence on the Random Forest or XGBoost prediction. This is most likely due to KNN being a neighbor-based model which could find broad efficiency metrics to be useful in distinguishing different teams.

The next most important feature was IS_Home. This feature was a binary variable in which it stated the home or away team. The home team would get a slight boost in winning percentage due to home court advantage. The K-nearest neighbors and XGBoost models valued this feature immensely more than the Random Forest models. This is most likely due to the neighbor based and gradient boosting classifiers valuing binary features more than a tree-based classifier. There is a possibility of some overfitting occurring here, which may need some further exploration in the future.

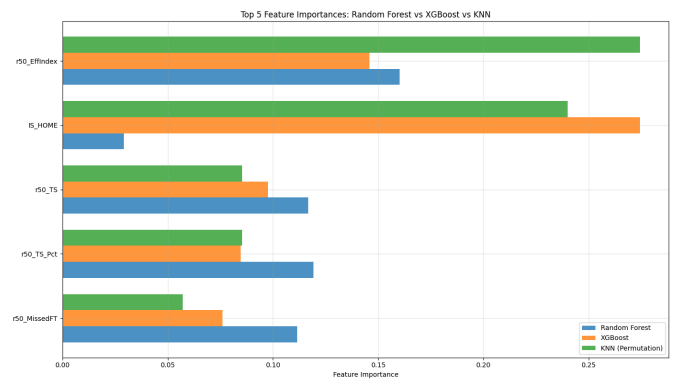


Figure 10: Top five feature importances comparison

VII: CONCLUSION / FUTURE WORK

As the world of sports becomes more driven by analytics, the future of NBA predictions may change a lot. One idea that we would love to work on in the future is adding player tracking data to our predictions. Player tracking data takes into account where all ten players are on the court at any given moment. This allows you to see how close a player is to contesting another player's shot, how they position themselves when going up for a rebound, and how well players are able to shoot depending on where the nearest defender is to them on the floor. This would allow us to see how two teams matchup more than before because if team A contests shots at a high rate, but team B shoots over contested shots above average we would assume their FG% would be higher than most teams vs team A and this might give them a slight edge in our model where traditional stats would not. This type of data is currently being used in MLB and the NFL, but is still a work in progress in the NBA so it is hard to find. When this data does become available expect a large change to how NBA games are predicted.

Another idea that we would love to add to our NBA prediction model is live play-by-play data. As games are played predictions may change. Betting markets are currently at the top of this market as they have implemented live betting to their platforms. Adding live play-by-play data would allow users to create predictions in the middle of games to see which team the model thinks would win given the score and how much time is remaining in the game. Coaches could also use play-by-play data to see which players are affecting the game more than others and adjust their lineups accordingly to create the best advantage for their team to win the game. This data is hard to come by so we would need to find a source that is fast and reliable. We would also need a model with a fast run time because the pace of an NBA game is extremely fast and the score can change multiple times in only a minute.

A third idea that we believe could help our model perform better is an up to date injury report. In the NBA, and all sports, injuries can drastically impact the results of a game. As we are writing this paper, the Indiana Pacers are currently 1-12 after being in the finals last season. This is largely due to the fact that their star player Tyrese Haliburton is injured. Since our model is predicting off the last 50 games it would not take that injury into account and it would still believe that the Pacers are one of the best teams in the league. If we are able to implement an injury report we could take the injuries into account in our predictions and allow the model to adjust its predictions. We would want the model to take into account how impactful each player is, so we would also need to find a dataset that keeps track of players statistics throughout the season so the model would not treat all players the same. This would be difficult to implement because NBA injuries occur all the time, and injury reports can be finalized up to a minute before the game begins. We would need to update the model throughout the day and also make sure that the data we are using is up to date as well.

Lastly, as mentioned in the Abstract section, we have not fully utilized the parameter tuning capabilities. The XGBoost model contains six different parameters that require extensive correlation testing in order to find the most accurate combination. Currently our parameters are set as follows:

- N_estimators = 200
- Max_depth = 6
- Learning_rate = 0.1
- Subsample = 0.8
- Colsample_bytree = 0.8
- Random_state = 42

To properly find the most reliable combination, we will need to perform excessive documentation of the performance associated with each combination. This will definitely be something that we plan on integrating in the near future.

Overall, there is still a lot that can be worked on to improve the model in the future. The NBA world will become more data driven and more research will be done into predicting

NBA games. We believe that there is a lot that can be worked on and we are excited to see what happens.

REFERENCES

- [1] Horvat, T., Job, J., Logožar, R., & Livada, Č. (2023). A Data-Driven Machine Learning Algorithm for Predicting the Outcomes of NBA Games. *Symmetry*, 15(4), 798. <https://doi.org/10.3390/sym15040798>
- [2] Wang, J. (2023). Predictive Analysis of NBA Game Outcomes Through Machine Learning <https://doi.org/10.1145/3635638.3635646>
- [3] Dong, H. (2024). Overview of the Application of Artificial Intelligence in Sports Events. 324–331. <https://doi.org/10.1145/3723936.3723986>